Innovation Centre for Education

NCS
NihonCommunicationSolutions

RABBI ZIDNI 'ILMA

YENEPOYA
(Deemed to be University)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design

On

Real-time Sentiment Analysis for Agriculture

SUBMITTED BY

Mohamed Suhaib U - 23BCDSBA13

BCA(Data Science and Big Data Analytycs)

UNDER THE GUIDANCE OF

Industry Mentor: sumit kumar shukla

Guided by: Raksha

TABLE OF CONTENTS

1. **Introduction…… 2**
   1.1. **Scope of the document……..3**
   1.2. **Intended Audience……5**
   1.3. **System overview……...6**
2. **System Design……..8**
   2.1. **Application Design……..……..8**
   2.2. **Process Flow……10**
   2.3. **Information Flow……..12**
   2.4. **Components Design……13**
   2.5. **Key Design Considerations……16**
   2.6. **API Catalogue……..17**
3. **Data Design……19**
   3.1. **Data Model……...19**
   3.2. **Data Access Mechanism……..21**
   3.3. **Data Retention Policies……21**
   3.4. **Data Migration……23**
4. **Interfaces……24**
5. **State and Session Management……25**
6. **Caching……26**
7. **Non-Functional Requirements……27**
   7.1. **Security Aspects……27**
   7.2. **Performance Aspects……28**
8. **References……..29**

1. Introduction

The Real-Time Sentiment Analysis for Agriculture system is a web-based analytical platform developed to process, analyze, and visualize customer feedback related to agricultural products and services. In the agriculture domain, customer reviews are valuable because they reveal product quality, effectiveness, usability, and field performance. Reviews on seeds, fertilizers, irrigation kits, pesticides, and agricultural tools often contain important insights that can help manufacturers, sellers, and farmers understand strengths and weaknesses of products.

Traditionally, such feedback is read manually, which is time-consuming and inefficient when the volume of reviews becomes large. Manual analysis also creates inconsistency because different people may interpret the same review differently. This project solves that problem by using Natural Language Processing techniques to automatically classify textual reviews into Positive, Negative, and Neutral sentiment categories.

The system is designed to work with agriculture-specific review datasets in CSV format. A user uploads a file or enters text, and the system processes the content, cleans the text, analyzes the sentiment, calculates statistical summaries, and displays results through an interactive dashboard. The platform combines backend processing in Python with a web-based frontend for visualization.

The project uses Pandas for CSV processing, NLTK for text preprocessing, TextBlob and VADER for sentiment scoring, Flask for backend communication, and HTML, CSS, JavaScript, and Chart.js for the frontend dashboard. This combination makes the system practical, extensible, and suitable for academic as well as small real-world use cases.

The HLD document explains the architecture and interaction of the major components without focusing on line-by-line implementation. It presents the system from a design perspective so that evaluators, developers, and students can understand how the application works as a complete solution.

1.1 Scope of the Document

This document provides the overall design of the Real-Time Sentiment Analysis for Agriculture system. It describes the major modules, their responsibilities, the system workflow, data movement, interfaces, and non-functional considerations. The purpose of this HLD is to give a clear blueprint of how the system is organized at a high level.

The document covers the complete end-to-end flow beginning from data input and ending with result visualization. It explains how a user can upload agricultural review data, how the backend validates and processes the file, how Natural Language Processing techniques are applied, how sentiment categories are generated, and how the output is displayed through graphs and summaries.

The scope also includes the structure of the API layer, the information exchanged between frontend and backend, the expected data model, and future extensibility of the system. Since the project is web-based, the HLD also includes a description of the presentation layer and how users interact with the dashboard.

The following major areas are covered in this document:

- Overall system architecture and layered design
- Workflow from user input to final sentiment output
- Major components and their responsibilities
- Data processing and visualization pipeline
- API catalog and interface structure
- High-level data design
- Performance and security considerations

The following items are outside the scope of this HLD:

- Detailed source code explanation
- Function-level implementation details
- Exact UI styling rules
- Advanced machine learning model training details
- Enterprise-scale deployment configuration
- Authentication and multi-user access control

This document is therefore meant to describe what the system is, how it is structured, and how its main parts work together.

1.2 Intended Audience

This document is intended for multiple types of readers.

The primary audience includes faculty members and academic evaluators who need to assess whether the project has been designed properly from an architectural perspective. For them, this document demonstrates that the project is not only implemented but also logically structured.

The second audience includes developers and future contributors. They may use this document to understand the system architecture before making changes or extending features. Since the design is modular, a developer can identify which component needs to be modified for a particular enhancement.

The third audience includes students and learners who want to understand how a sentiment analysis system is built in a real-world style. This HLD can serve as a reference for designing similar systems in healthcare, e-commerce, education, or other domains.

The fourth audience includes project stakeholders or mentors who want a conceptual understanding of the system without going into code-level

complexity. For them, the HLD acts as a simplified but professional summary of the solution.

Readers are expected to have a basic understanding of:

- Python programming
- Web applications
- CSV-based data handling
- Natural Language Processing concepts
- Client-server architecture

Even readers with limited technical background can use this document because the design is described in an understandable and structured manner.

1.3 System Overview

The Real-Time Sentiment Analysis for Agriculture system is a modular client-server application that analyzes agriculture-related reviews and presents sentiment insights visually. The system is designed for ease of use, so that even non-technical users can upload a CSV file and get meaningful results without manually reading each review.

The application accepts agricultural product reviews as input. These reviews may belong to categories such as seeds, fertilizers, pesticides, irrigation tools, pumps, or organic products. Once the file is uploaded, the system reads the contents and identifies the review text column. It then preprocesses the text by cleaning it and removing irrelevant words. After preprocessing, sentiment analysis is performed using a hybrid approach involving VADER and TextBlob.

The system classifies each review as Positive, Negative, or Neutral. In addition, it calculates polarity and subjectivity scores, summarizes counts and percentages, and groups results by category when category information is available. The output is displayed in the dashboard using bar charts, doughnut charts, polarity charts, histograms, and tables.

The system architecture is divided into major layers:

- Presentation Layer for user interaction and dashboard display
- Application Layer for handling requests and routing
- Sentiment Analysis Layer for text preprocessing and sentiment logic
- Data Layer for file input and processed review structures

The project is designed for near real-time execution. Once a file is uploaded, the analysis is completed quickly and results are displayed in the same interface. This makes it useful for demonstrations, academic evaluation, and lightweight analytics use cases.

The system can be extended in the future with features such as database storage, user authentication, multilingual review support, recommendation logic, machine learning classifiers, and cloud deployment.

2. System Design

2.1 Application Design

The application follows a layered and modular architecture. This design separates responsibilities and improves maintainability. Each layer has a specific role and communicates with adjacent layers through clearly defined data flow.

At a high level, the system is divided into four major layers.

The first layer is the Presentation Layer. This layer is responsible for user interaction. It provides the dashboard, the file upload area, the analysis button, the result cards, and all charts. It is built using HTML, CSS, and JavaScript. It also uses Chart.js for rendering visualizations.

The second layer is the Application Layer. This is implemented using Flask. It receives requests from the frontend, validates inputs, invokes the sentiment processing engine, and returns the results. This layer acts as the coordinator of the system.

The third layer is the Sentiment Analysis Layer. This is the core intelligence layer of the project. It reads reviews, cleans text using NLTK, computes polarity

using VADER, computes subjectivity using TextBlob, classifies the result, and prepares analytical summaries.

The fourth layer is the Data Layer. This includes input CSV files, temporary uploaded files, in-memory DataFrames, and processed review objects. In the current project, the main storage mechanism is file-based rather than database-based.

This architecture offers several advantages. It allows the user interface to change without affecting backend sentiment logic. It allows the backend API to evolve without requiring major modifications to frontend code. It also allows sentiment models to be improved independently.

The application design is intentionally modular because sentiment analysis projects often evolve with additional features. For example, a future machine

learning model can replace or augment the current sentiment engine while leaving the rest of the application intact.

2.2 Process Flow

The system follows a clear process flow from input to output. This process is sequential and easy to understand, which improves both reliability and traceability.

The process begins when the user opens the web application. The homepage provides options to upload a CSV file containing agricultural reviews. Once the file is selected, the user initiates the analysis process.

The backend first validates the file. It checks whether a file is present, whether the file has a CSV format, and whether it contains a valid review column. If validation fails, the system returns an error message.

If the file is valid, the backend reads it using Pandas and converts it into a DataFrame. The system then identifies important columns such as review, product, and category. If optional fields are missing, default values may be assigned.

The review text is then passed to the preprocessing stage. In this stage, the text is normalized by converting it to lowercase, removing punctuation, tokenizing words, and eliminating stopwords where required. This reduces noise and prepares the text for sentiment scoring.

After preprocessing, the sentiment analysis engine evaluates each review. VADER computes a polarity compound score, while TextBlob provides subjectivity information. The system then applies classification rules based on polarity thresholds to label the review as Positive, Negative, or Neutral.

Once all reviews are classified, the system aggregates results. It calculates total counts, percentages, category-wise breakdown, and polarity distribution buckets. It also prepares detailed review records for table display.

The final processed result is then sent back to the frontend in JSON format. The dashboard receives this response and renders cards, charts, tables, and summary insights. The user can then explore the output visually.

This flow ensures that the application remains structured, fast, and easy to debug.

2.3 Information Flow

The information flow describes how data moves between components and how it is transformed at each step.

The first stage of information flow begins at the user interface. The user uploads a CSV file using the browser. The file is transmitted to the Flask backend through an HTTP POST request.

The backend receives the uploaded file as form-data. It stores the file temporarily in the uploads directory and passes the file path to the sentiment analysis engine.

The sentiment engine reads the CSV file into a Pandas DataFrame. At this point, raw review data exists in tabular form. The engine then identifies the review-related field and optional supporting fields such as product and category.

Next, the raw review strings are transformed into cleaned text representations. This preprocessing stage reduces noise and improves consistency. The cleaned text is then sent to the sentiment analysis logic.

The sentiment logic generates multiple outputs for each review:

- polarity score
- subjectivity score
- sentiment label
- processed category grouping
- review-level output record

After row-level analysis, aggregate information is generated. This includes summary counts, sentiment percentages, category-wise distribution, histogram bucket distribution, and table-friendly records.

These results are then structured into a JSON response. The Application Layer returns this JSON response to the frontend.

The frontend JavaScript receives the response and uses the data to populate visual elements such as KPI cards, bar charts, doughnut charts, product polarity plots, histograms, scatter plots, and review tables.

Thus, the information flow can be summarized as:

User Input → Flask Request Handling → CSV Parsing → NLP Preprocessing → Sentiment Scoring → Classification → Aggregation → JSON Response → Dashboard Rendering

This flow is simple enough for maintainability yet rich enough to support multiple visual outputs.

2.4 Components Design

The system is composed of several major components, each contributing to a specific part of the overall workflow.

User Interface Component

This component handles all user interactions. It provides the dashboard layout, upload section, file preview, buttons, charts, and review table. The UI is designed to be modern and user-friendly so that users can upload a file and understand the results easily. It is implemented mainly in the frontend HTML

page and styled with CSS. JavaScript is used for dynamic interactions and rendering. K

File Handling Component

This component manages file selection, drag-and-drop upload behavior, format validation, and preview logic. It ensures that only CSV files are accepted and that invalid uploads are rejected with appropriate messages.

Backend Routing Component

This component is implemented in Flask and is responsible for defining routes such as the home page and analysis endpoint. It receives incoming requests, validates them, saves uploaded files temporarily, invokes the analysis engine, and returns responses.

Sentiment Engine Component

This is the most important analytical component of the system. It is responsible for column detection, text cleaning, sentiment scoring, classification, and aggregation. It acts as the core decision-making layer.

Data Processing Component

This component uses Pandas to read and manipulate CSV data. It handles row filtering, column insertion, grouping, and statistical summarization. It prepares the data in a way that is useful for the analysis engine and the dashboard.

Visualization Component

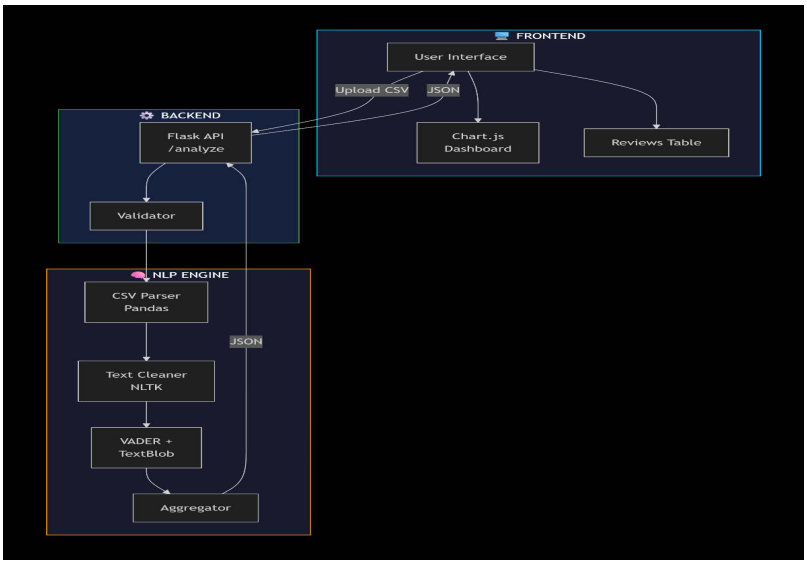
This component is implemented in frontend JavaScript using Chart.js. It receives structured data and produces charts such as bar charts, doughnut charts,

scatter plots, histograms, and horizontal product sentiment graphs. It also updates KPI cards and detailed review tables.

Report Generation Component

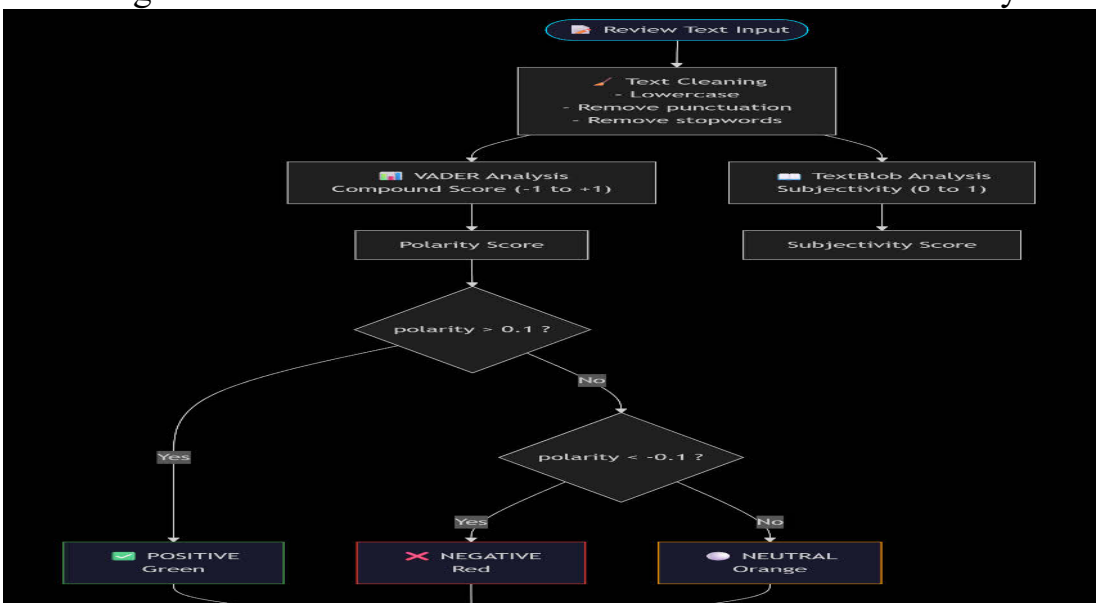
This component allows the generation of a simple PDF-style export from the dashboard by using HTML-based report content. It focuses on producing a readable summary of highly positive and highly negative items.

Together, these components form a cohesive end-to-end system. Their separation into modules makes the application easy to understand and easy to extend.



2.5 Key Design Considerations

Several design considerations influenced the final architecture of the system.



One important factor was usability. The system needed to be easy for non-technical users. Therefore, the interface was designed around a simple upload-and-analyze model rather than requiring manual parameter tuning.

Another important factor was modularity. Sentiment analysis, file handling, API routing, and visualization were kept separate so that changes in one module would not heavily affect the others.

Performance was also considered. The application uses efficient libraries such as Pandas and VADER, which are suitable for quick processing of CSV datasets. Chart rendering is performed in the browser, reducing backend burden.

Flexibility was another design priority. The system attempts to detect review, product, and category columns automatically. This helps users analyze different CSV file formats without strict dependency on one schema.

Agriculture domain relevance was also a key consideration. The VADER lexicon was extended with agriculture-specific words such as yield, germinate, fertile, burned, pest, wilted, and toxic. This improves classification relevance for agricultural product reviews.

Error handling and resilience were also considered. Invalid files, missing review columns, empty reviews, and other issues are handled gracefully through error responses and validation logic.

These considerations make the system academically strong and practically meaningful.

2.6 API Catalogue

The application exposes a small but useful set of routes.

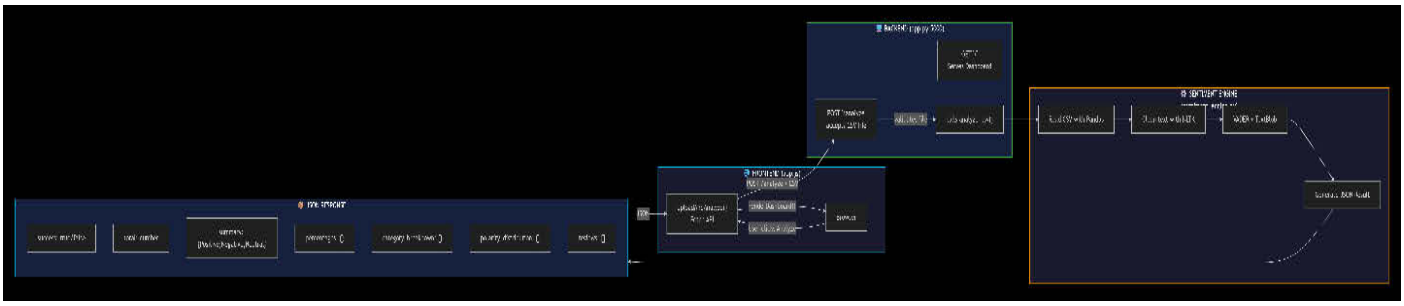
The home route loads the main dashboard interface. It serves as the entry point for the user.

The analyze route accepts uploaded CSV files through a POST request. It validates the input, stores the file temporarily, triggers the sentiment engine, and returns the structured result.

The frontend expects a JSON response containing:

- total review count
- sentiment summary counts
- sentiment percentages
- category-wise breakdown
- polarity distribution buckets
- review-level detailed records
- detected column names

This route is central to the application because it powers the entire analysis flow.



At a high level, the API design is intentionally simple. Since the project is academic and focused on sentiment analysis rather than large-scale platform design, the API avoids unnecessary complexity. However, it is still modular enough to be extended with more routes in the future, such as:

- summary-only analysis
- product ranking route

- downloadable report route
- history route
- authentication route

This API structure is appropriate for the current system scope.

3. Data Design

3.1 Data Model

The system uses a straightforward data model centered around review records.

Each input review generally includes:

- an identifier
- a product name
- a review text
- a category label

When the CSV does not include all of these, the system can still function by detecting the review column and generating defaults for missing optional fields.

After processing, each review record is transformed into a richer output structure containing:

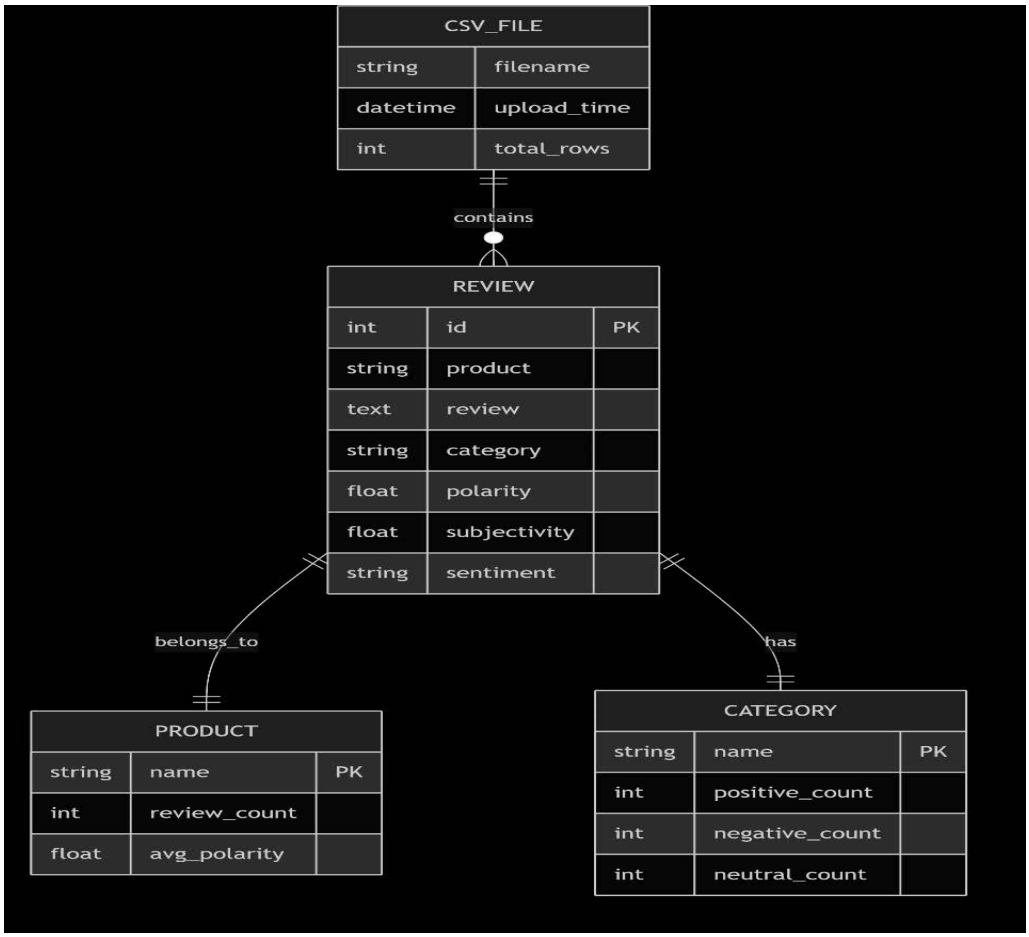
- id
- product
- review
- category
- polarity
- subjectivity
- sentiment

In addition to row-level data, the system also creates aggregate structures:

- summary counts for Positive, Negative, and Neutral

- percentage distribution
- category-wise grouped sentiment breakdown
- polarity histogram buckets
- lists for chart plotting

This hybrid data design supports both detailed review inspection and high-level managerial analysis.



3.2 Data Access Mechanism

The system is file-driven rather than database-driven. Input data is read from CSV files uploaded by the user.

The backend uses Pandas to access and process the data. Uploaded files are stored temporarily in an uploads folder, analyzed, and removed after processing. This approach reduces storage complexity and keeps the project lightweight.

Within the application lifecycle, the main data object is a Pandas DataFrame. This in-memory structure is used for:

- filtering rows
- grouping by category
- calculating distributions
- transforming reviews into JSON-compatible dictionaries

The frontend accesses processed data through JSON responses returned by the Flask backend. There is no direct database query from the UI. All dashboard values are computed and delivered by the backend analysis route.

This access pattern is appropriate for academic projects and small to medium batch datasets.

3.3 Data Retention Policies

In the current implementation, the system does not permanently store every analysis result in a database. Uploaded files are processed temporarily and removed after the analysis is complete.

This means the application follows a short-lived data retention approach. The purpose of this design is to keep the system simple, lightweight, and privacy-friendly.

The processed results remain available in the frontend only during the active session after the response is returned. Once the page is refreshed or a new file is uploaded, the previous analysis is replaced.

This approach has both benefits and limitations. It reduces the risk of unnecessary data persistence and simplifies storage management, but it also means the application does not maintain a long-term history of analyses.

In a future enterprise version, data retention policies could include:

- storing previous uploaded analyses
- keeping timestamped results
- archiving product-wise sentiment history
- adding user-level access control for past reports

For the current scope, temporary retention is sufficient.

3.4 Data Migration

Currently, the system is designed around CSV input and in-memory processing. There is no persistent relational or NoSQL database involved.

If the system is scaled in the future, data migration may involve moving from file-based input to a structured database such as MySQL, PostgreSQL, or MongoDB. In such a case, migration would include:

- defining a schema for reviews and results
- importing historical CSV data into the database
- updating backend functions to read from database queries
- adding indexing for faster search and aggregation
- redesigning retention and reporting policies

Another migration path could involve moving from a rule-based sentiment engine to a machine learning or transformer-based model. This would require updating the processing layer while leaving much of the application layer and presentation layer unchanged.

Thus, the current design is migration-friendly because the system components are already separated logically.

4. Interfaces

The system mainly provides a web-based user interface.

Users interact with the dashboard through their browser. The upload section allows file selection and drag-and-drop behavior. Buttons allow the user to initiate analysis, clear the selected file, reset the dashboard, or generate a report summary.

The backend interface is exposed through Flask routes. Communication between the UI and backend occurs using HTTP requests.

The output interface consists of:

- sentiment summary cards
- bar chart
- doughnut chart
- category chart
- polarity product chart
- histogram
- scatter plot
- review detail table

The system does not currently expose third-party public APIs, but its internal route structure is sufficient for the frontend application.

Future interface extensions may include:

- mobile app integration
- public REST API
- admin panel

- downloadable Excel report export
- multilingual input interface

5. State and Session Management

The application is mostly stateless from the server perspective. Each analysis request is handled independently.

When a user uploads a CSV file, the server processes it and immediately returns the result. The server does not maintain user-specific session state for analysis history.

On the frontend, temporary state exists in JavaScript. The dashboard stores the full set of processed reviews in browser memory so that filtering and chart rendering can happen dynamically. For example, the table filter buttons can display only Positive, Negative, or Neutral rows without sending another request to the server.

This state is session-local and disappears when the page is refreshed or reset.

This lightweight state model is suitable for the current academic project because it simplifies design and reduces server-side storage complexity.

6. Caching

The current system does not implement explicit caching.

Because the application processes uploaded files on demand, each request is analyzed freshly. This ensures that results always reflect the current uploaded dataset.

However, future caching opportunities exist. For example:

- commonly used NLP resources can be loaded once at application startup
- frequently repeated analysis of identical files can be cached temporarily
- frontend chart rendering states can be reused when only filters change
- product-level aggregated results can be memoized for faster redraw

At the present scale, caching is not essential because performance is already acceptable for small to medium datasets. Still, acknowledging this area strengthens the design documentation.

7. Non-Functional Requirements

7.1 Security Aspects

The system includes basic security measures appropriate for the current project scope.

File type validation ensures that users upload only CSV files. Invalid file formats are rejected before analysis begins.

Temporary file handling reduces long-term storage risk because uploaded files are deleted after processing.

The backend uses controlled route handling and does not expose sensitive system functionality publicly. Input validation helps prevent malformed data from entering the processing pipeline.

Since the system does not currently implement user authentication, role-based access control, or encrypted persistent storage, it is not intended for sensitive production environments in its current form.

If the system is expanded in the future, important security improvements would include:

- user authentication
- HTTPS deployment

- audit logging
- database encryption
- secure report access
- stricter file sanitization

For an academic agriculture review analysis system, the current level of security is acceptable.

7.2 Performance Aspects

The system is designed to provide quick feedback after file upload.

Pandas enables efficient CSV reading and grouping. VADER and TextBlob are lightweight compared to large transformer models, allowing faster processing. Frontend chart rendering is handled in the browser, which spreads computational work between backend and client.

Performance is good for small and medium review datasets. The system can process a few hundred or a few thousand records without heavy delay in typical academic or demo environments.

To maintain responsiveness, the current design avoids unnecessary database writes and limits complexity in each request. The modular structure also makes it easier to optimize specific bottlenecks later.

Future performance improvements could include:

- background task processing for large files
- batch optimization of preprocessing
- caching repeated calculations
- database indexing if persistent storage is added
- replacing synchronous workflows with queued tasks

Overall, the system performance is suitable for project demonstrations and lightweight analytics use.

10. References

- Pandas Documentation (pandas.pydata.org)Used for reading CSV files, data manipulation, and DataFrame operations
- NLTK Documentation (nltk.org) Used for text preprocessing such as tokenization and stopword removal
- TextBlob Documentation (textblob.readthedocs.io)Used for calculating sentiment polarity and subjectivity
- VADER Sentiment Analysis (github.com/cjhutto/vaderSentiment) Used for rule-based sentiment classification with domain-specific words
- Flask Documentation (flask.palletsprojects.com) Used for building backend API and handling file uploads
- Chart.js Documentation (chartjs.org)Used for creating interactive charts in the dashboard
- Python Official Documentation(docs.python.org)Core programming language reference
  Kaggle (kaggle.com)Used for sample datasets and understanding real-world data
- Stack Overflow (stackoverflow.com) Used for debugging and resolving coding issues
- Matplotlib Documentation (matplotlib.org)Used for generating charts and visualizations